THƯƠNG MẠI ĐIỆN TỬ VÀ ỨNG DỤNG

Bảo mật trong TMĐT

Giảng viên: Hoàng Mạnh Hà

Khoa Công nghệ thông tin – Trường Đại học Sài Gòn

hmha@sgu.edu.vn

Tổng quan về Bảo mật TMĐT

Tổng quan

- Bruce Schneier: "*Security is not a product, but a process*"
- Bảo mật không chỉ là cài đặt phần mềm chống virus hay tường lửa.
  - Trong TMĐT, bảo mật là việc bảo vệ **Dòng tiền** và **Dữ liệu**.
- Sinh viên IT cần tư duy bảo mật ngay từ khâu thiết kế Database (Security by Design).

Vấn đề "sống còn" với TMĐT

- **Thiệt hại tài chính trực tiếp:** Hacker lợi dụng lỗi logic để mua hàng với giá 0 đồng hoặc thay đổi số dư trong ví điện tử.
- **Chi phí khắc phục:** Ngừng hệ thống để điều tra, chi phí bồi thường cho khách hàng bị lộ thông tin.
- **Tác động pháp lý:** Theo Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân, doanh nghiệp có thể bị phạt nặng nếu để lộ thông tin khách hàng.
- **Hiệu ứng dây chuyền:** Một sự cố nhỏ làm xói mòn niềm tin, dẫn đến việc sụt giảm Traffic và Conversion Rate.

Tam giác bảo mật CIA

Tam giác bảo mật CIA

- **Confidentiality (Tính bảo mật):** Mã hóa mật khẩu (Bcrypt/Argon2), phân quyền (RBAC - Role Based Access Control).
  - *Ví dụ:* Nhân viên kho không được xem số dư tài khoản của khách.
- **Integrity (Tính toàn vẹn):** Sử dụng Checksum, Digital Signature (HMAC).
  - *Ví dụ:* Khi đơn hàng được chuyển từ Website sang VNPAY, số tiền vnp_Amount phải giữ nguyên 100%.
- **Availability (Tính sẵn sàng):** Load Balancing, chống DDoS, Auto-scaling trên Cloud.
  - *Ví dụ:* Website không được "sập" vào ngày Sale

Các "Tài sản" cần bảo vệ

- **Nhóm dữ liệu định danh (Personal Identifiable Information):** Họ tên, Số điện thoại, Email, Địa chỉ giao hàng.
- **Nhóm dữ liệu tài chính:** Lịch sử thanh toán, Mã giao dịch (TxnRef), Token cổng thanh toán.
- **Nhóm logic nghiệp vụ:** Công thức tính khuyến mãi, giá vốn sản phẩm, thuật toán Recommendation.
- **Nhóm hạ tầng:** File cấu hình .env, Access Key của AWS/Google Cloud, API Key của MoMo/VNPAY.

Phân tích các mối đe dọa

- **Xâm nhập dữ liệu (Injection Attacks):**
  - *SQL Injection:* Hacker nhập ' OR 1=1 -- vào ô tìm kiếm sản phẩm để bypass đăng nhập hoặc xóa bảng Orders.
- **Tấn công vào luồng thanh toán (Payment Logic Flaw):**
  - *Parameter Tampering:* Khách hàng dùng công cụ như Burp Suite để sửa giá sản phẩm từ 500.000đ thành 5.000đ trước khi nhấn "Thanh toán".
- **Đánh cắp phiên làm việc (Session Hijacking):** Chiếm quyền Admin thông qua Cookie không được bảo mật (thiếu flag HttpOnly).
- **Phản hồi giả mạo (Fake IPN - Instant Payment Notification):** Kẻ tấn công giả danh Server của MoMo gửi thông báo "Thanh toán thành công" tới Server của GreenLife để kích hoạt quy trình giao hàng.

Các nguyên tắc bảo mật cơ bản

- **Nguyên tắc đặc quyền tối thiểu (Least Privilege):** Database User của Web chỉ có quyền SELECT/INSERT/UPDATE, không được có quyền DROP table.
- **Defense in Depth (Phòng thủ đa tầng):** Bảo mật ở Web (HTTPS) -> Server (Firewall) -> App (Validation) -> Database (Encryption).
- **Fail-safe defaults:** Mặc định từ chối (Deny by default), chỉ cho phép khi đã xác thực.
- **Input Validation:** "Mọi dữ liệu từ người dùng đều là độc hại" – Phải sanitize dữ liệu trước khi xử lý.

Transport Security

Nguy cơ khi truyền tải dữ liệu

- **Vấn đề:** Trong môi trường Internet, dữ liệu đi qua rất nhiều trạm trung gian (Router, ISP).
- **Giao thức HTTP (Port 80):** Dữ liệu gửi đi dưới dạng **Plain text** (văn bản thuần túy).
  - *Hậu quả:* Hacker có thể dùng các công cụ như Wireshark để đọc trộm Password hoặc số thẻ thanh toán của khách hàng.
- **Tấn công Man-in-the-Middle (MITM):** Hacker đứng giữa, không chỉ đọc mà còn có thể **sửa đổi** gói tin (VD: sửa đổi địa chỉ nhận hàng).

Giải pháp HTTPS và Giao thức SSL/TLS

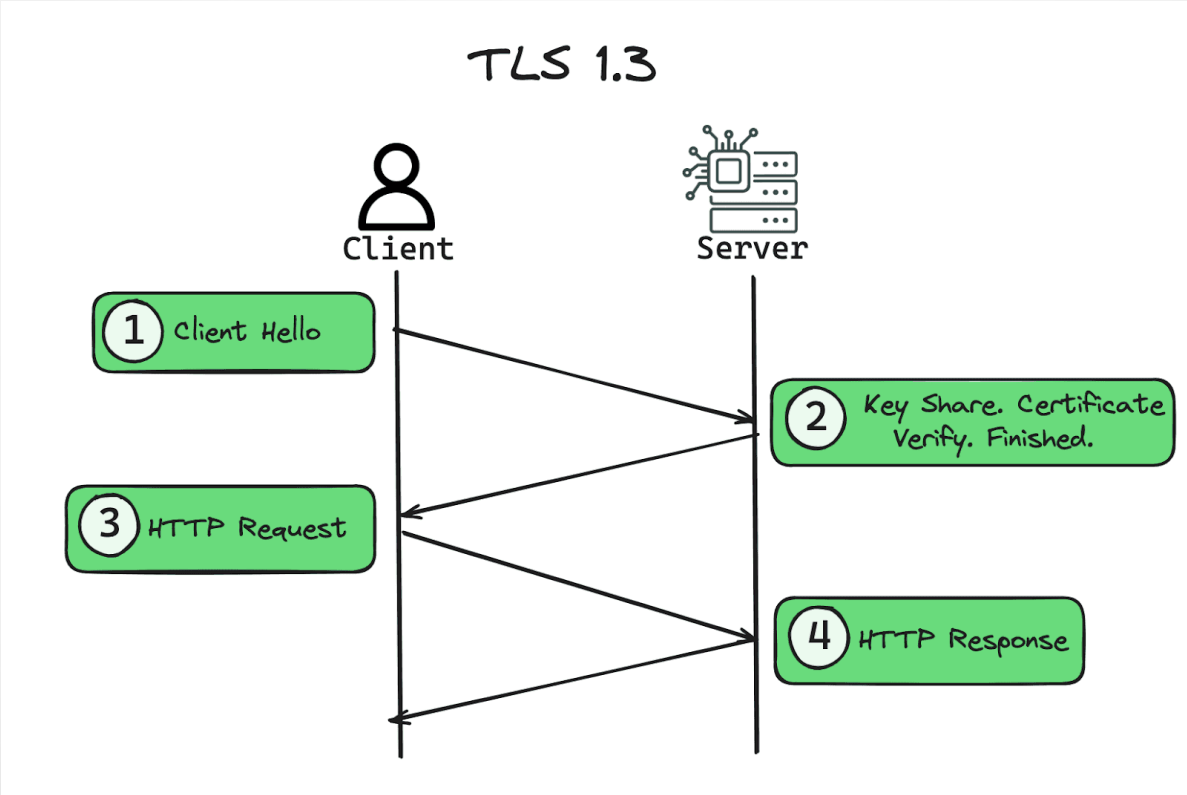
- **Khái niệm:** HTTPS = HTTP + SSL/TLS (Mã hóa).
- **Vai trò:**
  - **Mã hóa (Encryption):** Ẩn giấu dữ liệu để kẻ xấu không đọc được.
  - **Xác thực (Authentication):** Đảm bảo khách hàng đang kết nối đúng với Server của GreenLife, không phải một trang web giả mạo.
  - **Toàn vẹn (Integrity):** Đảm bảo dữ liệu không bị thay đổi trên đường truyền.
- **Thực tế:** Trình duyệt sẽ hiện biểu tượng "Ổ khóa" và cảnh báo "Not Secure" nếu website chỉ dùng HTTP.

Cơ chế TLS

- **Client:** Trình duyệt gửi các thuật toán mã hóa nó hỗ trợ.
- **Server & Certificate:** Server gửi **Public Key** và **SSL Certificate**.
- **Authentication:** Trình duyệt kiểm tra chứng chỉ với các tổ chức uy tín (CA).
- **Key Exchange:** Hai bên thống nhất một **Session Key** (khóa phiên) dùng để mã hóa đối xứng cho toàn bộ phiên làm việc.
- **Lưu ý:** Mã hóa bất đối xứng (Public/Private Key) chỉ dùng để "làm quen", còn truyền dữ liệu thật sự dùng mã hóa đối xứng để tối ưu tốc độ.

TLS 1.2

- https://supertokens.com/blog/what-is-tls



Chứng chỉ số

- **Thành phần của Certificate:** Thông tin chủ sở hữu, Public Key, Chữ ký số của tổ chức cấp phát (CA), Ngày hết hạn.
- **Các loại SSL phổ biến:**
  - **DV (Domain Validated):** Xác thực tên miền (phù hợp cho Đồ án).
  - **OV (Organization Validated):** Xác thực doanh nghiệp.
  - **EV (Extended Validation):** Xác thực chuyên sâu (hiện thanh địa chỉ xanh).
- **Công cụ hỗ trợ Đồ án:** Có thể nghiên cứu sử dụng **Let's Encrypt** (Miễn phí) hoặc các dịch vụ hỗ trợ sẵn trên Hosting/Cloud.

Ứng dụng vào đồ án

- Tất cả API Endpoint (đặc biệt là Login và Checkout) phải chạy trên HTTPS.
- **Cấu hình môi trường Local:**
  - Sử dụng **Ngrok** để tạo Tunnel HTTPS khi demo tích hợp VNPAY/MoMo (vì các cổng này yêu cầu ReturnURL/IPN (Instant Payment Notification) phải là URL công khai và an toàn).
  - Thiết lập Redirect tự động từ HTTP sang HTTPS trong Web Server (Nginx/Apache hoặc Middleware của Node.js/ASP.NET).

Chữ ký số và Cơ chế Checksum

HTTPS là chưa đủ

- **Kịch bản tấn công:**
  - Khách hàng thực hiện đơn hàng 1.000.000đ.
  - Tại trình duyệt, trước khi Redirect sang VNPAY, khách hàng (có kiến thức IT) dùng Tool can thiệp vào tham số trên URL, sửa vnp_Amount=1000000 thành vnp_Amount=1000.
  - Nếu không có chữ ký số, VNPAY chỉ nhận con số 1.000đ và cho phép thanh toán.
- HTTPS chỉ bảo vệ dữ liệu **trên đường truyền**. Chúng ta cần một cơ chế để **xác thực nội dung dữ liệu** không bị thay đổi ở hai phía đầu cuối

→ **Checksum/Digital Signature**

Hàm băm (Hash Function)

- Là thuật toán chuyển một chuỗi dữ liệu đầu vào (bất kể độ dài) thành một chuỗi ký tự có độ dài cố định.
- **Các đặc tính quan trọng:**
  - **Tính một chiều:** Không thể suy ngược từ mã băm ra dữ liệu gốc.
  - **Tính duy nhất (Hạn chế xung đột):** Hai dữ liệu khác nhau (dù chỉ 1 dấu phẩy) sẽ cho ra hai mã băm hoàn toàn khác nhau.
  - **Tốc độ:** Tính toán cực nhanh.
- **Các thuật toán phổ biến:**
  - SHA-256 (MoMo dùng)
  - SHA-512 (VNPAY dùng)

HMAC

- HMAC (**Hash-based Message Authentication Code**)
- **Vấn đề:** Nếu chỉ dùng hàm băm thông thường, Hacker có thể sửa dữ liệu rồi tự băm lại mã mới.
- **Giải pháp HMAC:** Kết hợp dữ liệu với một **Secret Key** (Khóa bí mật) trước khi băm.
- **Cơ chế:** Signature = Hash (Data + Secret Key)

➔ Chỉ có Website và Cổng thanh toán (VNPAY) mới biết Secret Key này. Hacker không có Key nên không thể tạo ra chữ ký hợp lệ cho dữ liệu đã bị sửa đổi.

Quy trình 4 bước tạo chữ ký số

1. **Thu thập dữ liệu:** Tập hợp tất cả các tham số của đơn hàng (Mã đơn, số tiền, nội dung...).
2. **Sắp xếp (Sorting):** Sắp xếp các tham số theo thứ tự **Alphabet của tên tham số** (Rất quan trọng - sai bước này sẽ sai chữ ký).
3. **Nối chuỗi (Concatenation):** Tạo thành một chuỗi Query String (VD: amount=1000&orderId=123...).
4. **Băm với Secret Key:** Sử dụng thuật toán (VD: HMAC-SHA512) và Secret Key được cấp trong Sandbox để tạo ra chuỗi Hash cuối cùng.

Quy trình kiểm tra (Verify) tại Webhook/IPN

- *Khi Cổng thanh toán trả về kết quả cho Server:*
- **Bước 1:** Server nhận các tham số trả về + chuỗi chữ ký (gọi là vnp_SecureHash hoặc signature).
- **Bước 2:** Server của sinh viên thực hiện lại đúng quy trình 4 bước để tạo ra một "Chữ ký nội bộ".
- **Bước 3:** So sánh "Chữ ký nội bộ" với chữ ký nhận được từ Cổng thanh toán.
  - **Khớp:** Giao dịch tin cậy -> Cập nhật Database.
  - **Không khớp:** Giao dịch bị can thiệp/giả mạo -> Từ chối xử lý.

Các lỗi thường gặp

- **Sai Secret Key:** Dùng nhầm Key của MoMo cho VNPAY hoặc copy thiếu ký tự.
- **Sai thứ tự tham số:** Không thực hiện bước Sort Alphabet trước khi băm.
- **Lỗi định dạng số tiền:** Ví dụ VNPAY yêu cầu số tiền nhân với 100 (1000đ phải gửi là 100000)
- **Lỗi Encoding:** Các ký tự đặc biệt (khoảng trắng, tiếng Việt có dấu) không được xử lý đồng nhất giữa lúc băm và lúc gửi URL.

Mã giả cho hàm tạo chữ ký

// 1. Dữ liệu đơn hàng

data = { vnp_Amount: 100000, vnp_TxnRef: "DH001", ... }

// 2. Sắp xếp Alphabet theo Key

sortedData = sortKeysAlphabetically(data)

// 3. Tạo chuỗi ký tự (Data String)

rawString = buildQueryString(sortedData)

// 4. Tạo chữ ký HMAC-SHA512

secretKey = process.env.VNP_HASHSECRET

secureHash = HMAC_SHA512(rawString, secretKey)

// 5. Gắn vào URL thanh toán

paymentUrl = vnp_Url + "?" + rawString + "&vnp_SecureHash=" + secureHash

The Raw String Problem

- Chỉ cần sai một ký tự khoảng trắng hoặc kiểu dữ liệu là mã Hash sẽ khác hoàn toàn. VD **URL Encoding:**
  - Khi nối chuỗi, các ký tự đặc biệt (tiếng Việt, khoảng trắng, dấu @) phải được mã hóa đồng nhất.
  - *Ví dụ:* Nếu vnp_OrderInfo là "Thanh toan hoa don", hệ thống băm chuỗi có khoảng trắng nhưng khi gửi đi trình duyệt lại chuyển thành %20. Nếu phía nhận không giải mã ngược lại trước khi băm kiểm tra -> **Lỗi sai chữ ký.**
- **Kỹ thuật xử lý:** Luôn sử dụng các hàm chuẩn như
  - HttpUtility.UrlEncode (.NET)
  - urllib.parse (Python)
  - URLSearchParams (JS).

Tính lũy đẳng (Idempotency)

- Tính lũy đẳng: Kết quả không đổi dù thực hiện 1 lần hay 100 lần, trạng thái hệ thống chỉ thay đổi ở lần đầu tiên hoặc giữ nguyên sau lần đầu.
- **Vấn đề:** Cổng thanh toán có cơ chế gọi IPN nhiều lần (Retry) nếu Server không phản hồi RspCode=00 ngay lập tức.
- **Hậu quả:** Nếu không kiểm tra, mỗi lần IPN gọi đến, hệ thống lại cộng tiền vào tài khoản khách hoặc gửi email xác nhận đơn hàng → Gây thất thoát hoặc phiền hà.
- **Quy trình kiểm tra trạng thái đơn hàng:**
  - Trước khi cập nhật đơn hàng thành "Paid", phải kiểm tra xem trạng thái hiện tại trong Database có phải là "Pending" hay không.
  - Nếu đơn hàng đã là "Paid" hoặc "Failed" rồi -> Trả về kết quả cho Cổng thanh toán ngay, không xử lý lại logic nghiệp vụ nữa.

Kỹ thuật Debug chữ ký (Logging strategy)

- Làm đúng các bước vẫn báo sai chữ ký?

→ **Log chuỗi Raw String.**

- Ghi log (ghi ra file hoặc console) chuỗi cuối cùng sau khi đã sắp xếp và nối lại, ngay trước khi bỏ vào hàm băm.
- Sau đó, dùng chuỗi đó so sánh với công cụ **Check Hash Online** của VNPAY/MoMo cung cấp.
- Nếu chuỗi Log khác chuỗi của Cổng thanh toán dù chỉ 1 ký tự → xác định sai ở khâu nối chuỗi hay khâu sắp xếp.

Phân biệt Hashing vs. Encryption

Đặc điểm	Hashing (Băm)	Encryption (Mã hóa)
Tính chất	**Một chiều (One-way).** Không thể giải ngược để lấy lại dữ liệu gốc.	**Hai chiều (Two-way).** Có thể giải mã để lấy lại dữ liệu gốc nếu có Khóa.
Mục đích chính	**Tính toàn vẹn (Integrity).** Kiểm tra xem dữ liệu có bị sửa đổi không.	**Tính bảo mật (Confidentiality).** Giấu dữ liệu để người không có khóa không đọc được.
Độ dài kết quả	Luôn cố định (VD: SHA-512 luôn ra chuỗi 128 ký tự hex).	Thường thay đổi tùy theo độ dài dữ liệu gốc và thuật toán.
Thuật toán phổ biến	MD5, SHA-256, SHA-512, **HMAC**.	AES, RSA, DES, Blowfish.
Ứng dụng trong Đồ án	Tạo chữ ký vnp_SecureHash để xác thực đơn hàng; Lưu mật khẩu User.	Mã hóa thông tin thẻ (thường Cổng thanh toán làm); Mã hóa dữ liệu nhạy cảm trong DB.

Bảo mật tích hợp API & Webhook

Cơ chế Webhook (IPN)

- IPN (Instant Payment Notification) là một Webhook — cơ chế để Server của Cổng thanh toán gọi trực tiếp đến API của Server doanh nghiệp để thông báo trạng thái giao dịch.
- **Tại sao cần IPN?**
  - Người dùng có thể tắt trình duyệt ngay sau khi thanh toán thành công (không đợi Redirect về website).
  - IPN đảm bảo hệ thống cập nhật trạng thái đơn hàng ngay cả khi luồng Frontend bị ngắt.
- **Đặc điểm:** Đây là giao tiếp **Server-to-Server**, không qua trình duyệt của người dùng.

Các kịch bản tấn công vào Endpoint IPN

- **Giả mạo IPN (Spoofing):** Hacker tìm ra URL API nhận IPN của website (VD: /api/vnpay-ipn) và gửi dữ liệu giả báo "Thành công" để chiếm đoạt hàng hóa.
- **Tấn công phát lại (Replay Attack):** Hacker chặn một gói tin IPN "Thành công" thật, sau đó gửi lại nhiều lần vào Server nhằm mục đích gây sai lệch dữ liệu (nếu hệ thống không có tính Lũy đẳng).
- **Thay đổi Payload:** Hacker can thiệp vào gói tin để thay đổi số tiền hoặc mã đơn hàng (đã giải quyết bằng Chữ ký số ở Phần 3).

Xác thực nguồn gốc (IP Whitelisting)

- **Nguyên tắc:** Server chỉ chấp nhận yêu cầu IPN đến từ các địa chỉ IP xác định của đối tác (VNPAY/MoMo).
- **Kỹ thuật:** Kiểm tra Header của Request để lấy Client IP.
  - So sánh với danh sách IP Sandbox/Production mà Cổng thanh toán cung cấp trong tài liệu API.
  - **Lưu ý:** Nếu dùng Proxy/Load Balancer, cần kiểm tra Header X-Forwarded-For.
- **X-Forwarded-For (XFF)** là một tiêu đề HTTP tiêu chuẩn (standard HTTP header) được sử dụng để xác định địa chỉ IP gốc của máy khách (client) khi họ kết nối đến máy chủ web thông qua proxy HTTP hoặc bộ cân bằng tải (load balancer). Nó giúp máy chủ biết IP thực của người dùng, thay vì chỉ thấy IP của proxy trung gian.

Xác thực Logic nghiệp vụ (Cross-check)

- Khi nhận được IPN, không được cập nhật DB ngay mà phải kiểm tra 4 điều kiện:
  - **Kiểm tra Chữ ký (Signature):** Dùng thuật toán **Băm (HMAC-SHA512)** để xác thực dữ liệu gửi qua không bị sửa đổi.
  - **Kiểm tra Đơn hàng (Order Existence):** Mã đơn hàng vnp_TxnRef có tồn tại trong Database không?
  - **Kiểm tra Số tiền (Amount Validation):** Số tiền từ IPN gửi về có khớp với số tiền lưu trong đơn hàng ở DB không? (Tránh lỗi sửa giá).
  - **Kiểm tra Trạng thái (Status Check):** Đơn hàng có đang ở trạng thái "Chờ thanh toán" (Pending) không?

Đảm bảo tính Lũy đẳng (Idempotency)

- Nếu 2 yêu cầu IPN trùng lặp gửi tới cùng lúc, làm sao để đơn hàng không bị xử lý 2 lần?
- **Giải pháp Kỹ thuật:**
  - **Database Constraint:** Sử dụng Transaction (Giao dịch nguyên tử) với mức cô lập (Isolation Level) phù hợp.
  - **Atomic Update:** Sử dụng câu lệnh SQL có điều kiện: UPDATE Orders SET Status = 'Paid' WHERE Id = 'DH01' AND Status = 'Pending';
  - **Kết quả:** Nếu câu lệnh ảnh hưởng đến 1 dòng -> Thành công. Nếu ảnh hưởng đến 0 dòng (do đã được cập nhật trước đó) -> Báo thành công cho Cổng thanh toán nhưng không thực hiện logic tiếp theo (gửi mail, cộng tiền).

Session/Cookie truyền thống?

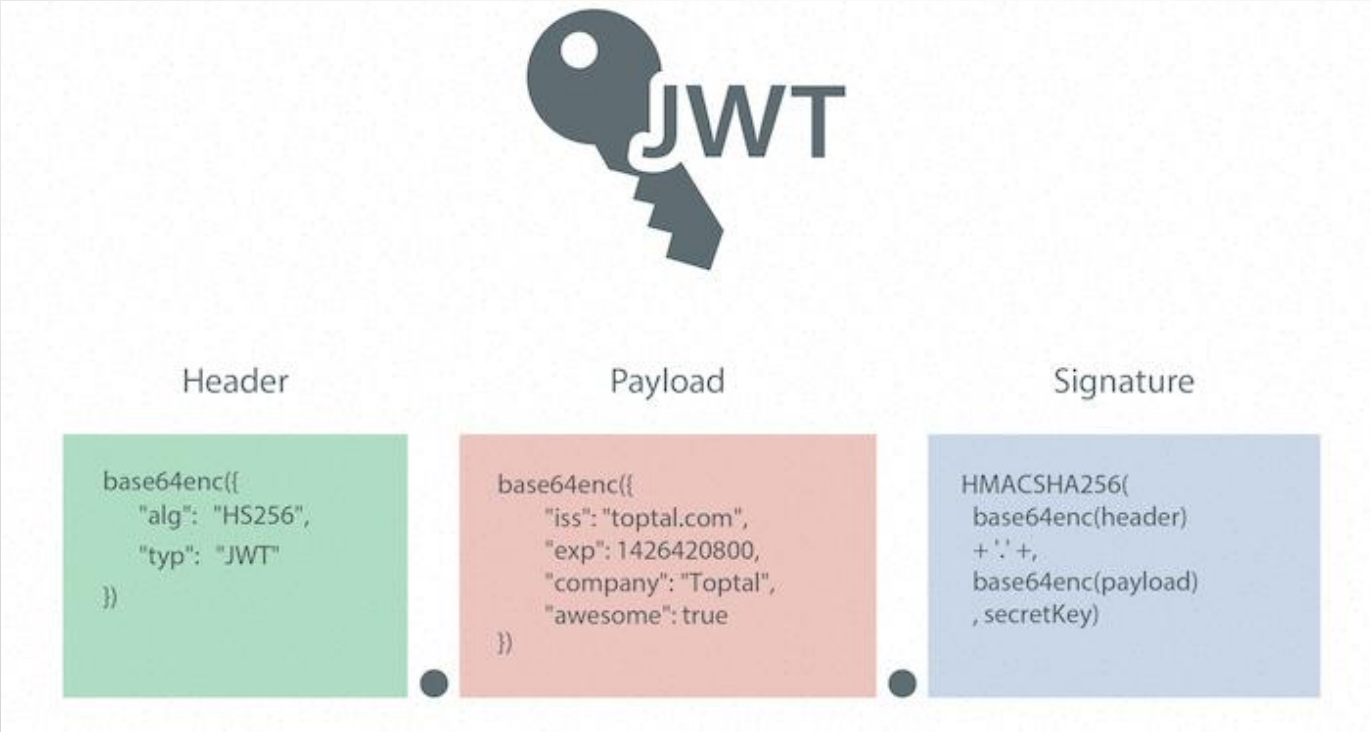
- **Vấn đề của Session/Cookie:**
  - **Stateful:** Server phải lưu trữ dữ liệu phiên làm việc trong bộ nhớ (RAM/Session Store), khó mở rộng (Scaling) khi có hàng ngàn khách hàng truy cập cùng lúc.
  - **Cross-Origin Resource Sharing (CORS):** Giao thức Cookie gặp nhiều rào cản khi Frontend và Backend nằm ở hai Domain/Sub-domain khác nhau.
- **Giải pháp Token-based (Stateless):**
  - Server không cần nhớ trạng thái đăng nhập của khách hàng.
  - Mọi thông tin xác thực nằm gọn trong một chuỗi ký tự (Token) được gửi kèm theo mỗi Request.
  - Phù hợp hoàn hảo cho kiến trúc Microservices hoặc ứng dụng Mobile.

Cơ chế JSON Web Token (JWT)

- **Khái niệm:** JWT là một chuẩn mở (RFC 7519) định nghĩa cách truyền tin an toàn giữa các bên dưới dạng đối tượng JSON.
- **Cấu trúc 3 phần của JWT (Ngăn cách bằng dấu .)**:
  - **Header:** Chứa loại token và thuật toán băm (ví dụ: HS256).
  - **Payload:** Chứa các thông tin cần thiết (Claims) như userId, role, exp (thời gian hết hạn).
  - **Signature (Chữ ký):** Được tạo ra bằng cách lấy (Header + Payload) kết hợp với một **Secret Key** thông qua hàm **Băm (Hashing)**.

JWT

- **Công thức tạo Chữ ký:**
- $Signature = HMACSHA256(base64(Header) + "." + base64(Payload), SecretKey)$



Luồng xác thực với JWT

- **Đăng nhập (Login):** Khách hàng gửi Username/Password lên Backend.
- **Tạo Token (Issue):** Backend xác thực đúng sẽ tạo một JWT (ký bằng Secret Key) và trả về cho Client.
- **Lưu trữ (Storage):** Client lưu Token vào localStorage hoặc Secure Storage (Mobile).
- **Gửi yêu cầu (Request):** Với các API cần bảo mật (như "Xem lịch sử đơn hàng"), Client gửi Token trong Header theo định dạng: Authorization: Bearer <token>
- **Xác minh (Verify):** Backend nhận Token, dùng Secret Key để **Băm** lại và so sánh chữ ký. Nếu khớp và chưa hết hạn -> Trả về dữ liệu.

Bảo mật Payload
Nguyên tắc "Cấm lưu mật khẩu"

- **Sai lầm thường gặp:** Bỏ mật khẩu (dù đã băm) hoặc thông tin nhạy cảm vào Payload của JWT.
- **Cảnh báo:** Payload chỉ được mã hóa theo chuẩn Base64 (dễ dàng giải mã ngược để đọc nội dung), **không phải là mã hóa (Encryption)** bảo mật. Bất kỳ ai có Token đều đọc được Payload.
- **Nguyên tắc thiết kế Payload:**
  - Chỉ lưu userId, username, permissions (quyền hạn).
  - Luôn có trường exp (Expiration Time): Token chỉ nên có hiệu lực trong thời gian ngắn (ví dụ: 1 giờ).
  - **Bảo mật phía Server:** Secret Key dùng để ký JWT phải được lưu trong file .env, tương tự như Secret Key của VNPAY.

Tính Lũy đẳng và Refresh Token

- **Vấn đề:** Nếu Token hết hạn giữa chừng khi khách đang mua hàng, làm sao để khách không phải đăng nhập lại?
- **Giải pháp: Refresh Token.**
  - Sử dụng một Token phụ (thời hạn dài hơn) để đổi lấy Access Token mới.
- **Kết hợp Tính Lũy đẳng (Idempotency):**
  - Khi Client gửi lệnh "Đặt hàng" kèm theo JWT. Nếu mạng lag, Client gửi lại lần 2 với cùng một Token và orderID.
  - Backend phải dùng orderID để kiểm tra trong DB trước khi xử lý, tránh tạo 2 đơn hàng cho cùng một yêu cầu.

Các lỗ hổng Web phổ biến

SQL Injection

- **Vấn đề:** Lập trình viên nối chuỗi trực tiếp từ dữ liệu người dùng vào câu lệnh SQL.
- **Nguy cơ:** Hacker có thể xóa bảng Orders, lấy toàn bộ danh sách Users hoặc Bypass (vượt qua) màn hình đăng nhập Admin.
- **Ví dụ mã độc hại:** * SELECT * FROM Users WHERE username = '' OR 1=1 --' AND password = '...'
- **Giải pháp: Tuyệt đối không nối chuỗi SQL.**
  - Sử dụng **Parameterized Queries (Prepared Statements)**.
  - Sử dụng các thư viện ORM (Sequelize, Eloquent, Entity Framework) đã tích hợp sẵn cơ chế chống Injection.

Broken Access Control

- **Vấn đề:** Người dùng có thể truy cập vào tài nguyên mà họ không được phép (VD: Khách hàng vào được trang Admin).
- **Lỗi IDOR (Insecure Direct Object Reference):**
  - Khách hàng A thay đổi URL từ "*/order/100*" thành "*/order/101*" và xem được đơn hàng của Khách hàng B.
- **Giải pháp:**
  - Luôn kiểm tra quyền sở hữu tài nguyên ở phía Server: SELECT * FROM Orders WHERE id = orderId AND customerId = currentUserId;
  - Áp dụng cơ chế **RBAC (Role-Based Access Control)** chặt chẽ.

Cross-Site Scripting (XSS)

- **Vấn đề:** Hacker chèn mã JavaScript độc hại vào website (thông qua các ô nhập liệu như Comment, Đánh giá sản phẩm).
- **Nguy cơ:** Khi Admin vào xem đánh giá, mã script sẽ chạy và đánh cắp **JWT/Session Token** lưu trong LocalStorage hoặc Cookie.
- **Giải pháp:**
  - **Sanitize Input:** Lọc bỏ các thẻ <script>, onclick,... từ dữ liệu người dùng gửi lên.
  - **Encode Output:** Chuyển đổi các ký tự đặc biệt thành HTML Entities (VD: < thành <) trước khi hiển thị.
  - Sử dụng Cookie với flag HttpOnly để JavaScript không thể đọc được Token.

Security Misconfiguration

- **Vấn đề:** Để lộ thông tin hệ thống qua cấu hình không an toàn.
- **Lỗi phổ biến:**
  - Để chế độ **Debug Mode: ON** trên môi trường Production (hiển thị toàn bộ cấu trúc thư mục, lỗi code khi gặp sự cố).
  - Không phân quyền thư mục (cho phép thực thi file trong thư mục Upload ảnh).
  - **Lộ file .env:** Hacker truy cập trực tiếp "*/.env*" để lấy Secret Key của VNPAY/MoMo.
- **Giải pháp:**
  - Chặn truy cập các file nhạy cảm trong cấu hình Web Server (Nginx/Apache).
  - Tắt chế độ Debug khi triển khai thực tế.

Thông tin liên hệ

- GV: Hoàng Mạnh Hà
- https://sites.google.com/view/hoangha84
- hmha@sgu.edu.vn

SCAN ME